

AI ENGINEERING FOR .NET ARCHITECTS

From LLM Foundations to Production Agentic Systems

A 5-Week Intensive Curriculum

For senior .NET developers and architects (10+ years) moving into applied AI engineering

Curriculum prepared August 2026

Course Overview

This course is built for a specific person: a .NET architect or senior developer with roughly a decade or more of production experience who already thinks in terms of clean architecture, dependency injection, distributed systems, and CI/CD — and who now needs to become genuinely capable with applied, production-grade AI. It does not re-teach C#, ASP.NET Core Web API, SQL Server, Docker, or Azure DevOps; it assumes fluency in all of them and spends every hour on what's actually new: generative AI, retrieval, agentic systems, and the operational discipline required to run them safely in production.

The program is organized in three phases — Foundations, Building, and Production — and is simultaneously project-driven: four progressively larger projects are built across the five weeks, each one folding in the technologies from that phase, culminating in a single deployed Capstone platform in Week 5.

Audience & Prerequisites

- Professional .NET development experience; comfortable in an architect or senior-engineer role
- Strong working knowledge of C#, ASP.NET Core Web API, Entity Framework/SQL Server, Docker, and Azure fundamentals
- Existing exposure to Azure DevOps/Azure Pipelines and basic CI/CD concepts
- No prior AI/ML experience required — this course starts from first principles on the AI-specific material
- Access to an Azure subscription with permission to provision Azure OpenAI / Microsoft Foundry resources

Format & Duration

Duration	5 weeks, 5 sessions per week (25 sessions total)
Daily commitment	~3 hours/day: 90 min instruction + 90 min hands-on lab
Delivery mode	Live/cohort-based or self-paced with checkpoint reviews
Total contact hours	~75 hours instruction/lab + capstone integration time
Cohort size (if live)	8–16 participants recommended for effective project reviews

Learning Outcomes

By the end of this course, participants will be able to:

- Design and build production ASP.NET Core services that consume LLMs through Microsoft.Extensions.AI, with provider-agnostic, testable AI service layers
- Build retrieval-augmented generation (RAG) systems grounded in enterprise data, using vector databases, data-ingestion pipelines, and Semantic Kernel
- Design, build, and safely orchestrate multi-agent systems with Microsoft Agent Framework and expose enterprise capabilities via the Model Context Protocol (MCP)
- Decide when classical ML (ML.NET) is the right tool instead of, or alongside, generative AI
- Evaluate AI system quality systematically, defend against prompt injection, and apply responsible-AI and content-safety controls
- Containerize, deploy, and operate AI applications on Azure with full CI/CD via Azure DevOps/Azure Pipelines, plus cost, latency, and quality monitoring

Technology Stack Covered

Category	Technologies
Core language & web	C#, ASP.NET Core Web API, Blazor/Razor (front end for labs)
AI abstraction layer	Microsoft.Extensions.AI (IChatClient, IEmbeddingGenerator), Microsoft.Extensions.VectorData, Microsoft.Extensions.DataIngestion
Model providers	OpenAI, Azure OpenAI, Microsoft Foundry, Foundry Local (on-device)
Orchestration frameworks	Semantic Kernel, Microsoft Agent Framework 1.0
Interoperability	Model Context Protocol (MCP) C# SDK v2.0
Retrieval & data	Vector databases (Azure AI Search, pgvector, Qdrant, SQL Server), RAG pipelines
Classical ML	ML.NET, Model Builder, ONNX
Quality & safety	Microsoft.Extensions.AI.Evaluation, Azure AI Content Safety, responsible-AI governance
Infrastructure	SQL Server, Docker, Azure Container Registry, Azure Container Apps/AKS
DevOps	Azure DevOps, Azure Pipelines, Bicep/Terraform (overview), Application Insights/Azure Monitor

Program Structure at a Glance

Phase	Week	Theme	Capstone Milestone
1 — Foundations	Week 1	Thinking like an AI engineer: Microsoft.Extensions.AI, providers, prompting & tool calling	Project 1: SmartSupport API
2 — Building	Week 2	Data, retrieval & RAG: vector databases, ingestion, Semantic Kernel	Project 2: Enterprise Knowledge Assistant
2 — Building	Week 3	Agentic AI: Microsoft Agent Framework, MCP, multi-agent orchestration	Project 3: Agentic Ops Assistant
3 — Production	Week 4	ML.NET, evaluation, responsible AI, cost & performance engineering	Checkpoint: all projects evaluated & hardened
3 — Production	Week 5	Containers, Azure deployment, CI/CD, monitoring	Capstone: unified platform deployed live

Capstone Project

The course culminates in a single Capstone: the AI-Augmented Enterprise Platform. Rather than a fifth, separate project, the Capstone integrates Projects 1–3 — the support API, the RAG knowledge assistant, and the agentic ops workflow — behind one deployed system, adds the ML.NET pre-filter from Week 4, and ships it through the full CI/CD pipeline built in Week 5, complete with live monitoring. This mirrors how these technologies actually get combined in real enterprise systems, and gives each participant a portfolio-ready, end-to-end reference architecture.

Detailed Curriculum

Phase 1 — Foundations (Week 1)

Goal: replace intuition about generative AI with an engineer's mental model, and ship a first production-shaped AI service.

Day 1 — Thinking Like an AI Engineer

Focus: Reframe generative AI in terms an architect already trusts — a remote dependency with a new failure mode, cost model, and latency profile, not magic.

Topics covered:

- How LLMs actually work at a level useful for engineering decisions: tokens, context windows, embeddings, temperature/top-p, non-determinism
- The .NET AI ecosystem map: Microsoft.Extensions.AI, Semantic Kernel, Microsoft Agent Framework, ML.NET, MCP, Microsoft Foundry — what each one is for and how they layer together
- Provider landscape: OpenAI vs Azure OpenAI vs Microsoft Foundry (cloud) vs Foundry Local (on-device) — cost, compliance, data-residency, and latency trade-offs an architect must weigh
- New non-functional requirements: hallucination, prompt injection, cost-per-request, model drift — mapped against the NFRs you already own (availability, security, performance)

Hands-on lab: Environment setup and smoke test: provision an Azure OpenAI (or Microsoft Foundry) resource, scaffold a .NET 10 console app, and make your first IChatClient call end-to-end, including secrets via .NET User Secrets / Key Vault.

Deliverable: Working dev environment (VS/VS Code, .NET SDK, Docker Desktop, Azure CLI) plus a documented decision matrix: OpenAI vs Azure OpenAI vs Foundry Local for three example scenarios from your own organization.

Day 2 — Microsoft.Extensions.AI — The Unifying Abstraction

Focus: Learn the IChatClient / IEmbeddingGenerator abstractions that Microsoft carved out of Semantic Kernel to become the standard, DI-friendly foundation for every layer above it.

Topics covered:

- IChatClient and IEmbeddingGenerator<TInput,TEmbedding>: the core contracts and why Microsoft standardized on them across the whole ecosystem
- Registering AI clients with dependency injection in ASP.NET Core, exactly like you already register HttpClient or a repository
- The middleware pipeline: chat client decorators for caching, logging/telemetry, rate limiting, and automatic function invocation
- Swapping providers without touching business logic: pointing the same IChatClient at OpenAI, Azure OpenAI, or a local model

Hands-on lab: Refactor Day 1's console app into a small ASP.NET Core minimal API. Register IChatClient via DI, add a logging + caching middleware decorator, and expose a /chat endpoint.

Deliverable: ASP.NET Core Web API project with a provider-agnostic AI service layer (interfaces, DI registration, middleware) that compiles against Azure OpenAI today and can be repointed with a config change.

Day 3 — Model Providers, Configuration & Microsoft Foundry

Focus: Go deep on where your model actually runs: Azure OpenAI provisioning, Microsoft Foundry as the managed model catalog and hosted-agent platform, and Foundry Local for on-device/offline scenarios.

Topics covered:

- Azure OpenAI resource provisioning, deployments, quota, and regional availability planning
- Microsoft Foundry: unified model catalog, hosted agents, and how it complements Microsoft.Extensions.AI as the app-facing layer
- Foundry Local GA: zero-dependency on-device inference (GPU/NPU/CPU auto-selection), the Microsoft.AI.Foundry.Local NuGet package, and the OpenAI-compatible local endpoint
- Configuration and secrets: appsettings + Key Vault + managed identity patterns for AI credentials in an enterprise .NET app

Hands-on lab: Add a Foundry Local fallback path to the Day 2 API: when Azure OpenAI is unreachable or a request is flagged low-sensitivity, route it to a small local model (e.g., a Phi model) running via Foundry Local, using the same IChatClient abstraction.

Deliverable: A provider-routing strategy (written + code) showing cloud vs local model selection logic, with a one-page architecture note on when each mode is appropriate for a regulated enterprise.

Day 4 — Prompting, Tool Calling & Structured Output for C# Developers

Focus: Treat prompts and function calling like the API contracts and interfaces you already design daily — with explicit schemas, versioning, and testability.

Topics covered:

- System/developer/user prompt roles, prompt templates, and versioning prompts like code
- Native function/tool calling from C#: exposing existing service methods as AI-callable tools with attributes and JSON schema generation
- Structured output: constraining responses to a C# record/DTO via schema-guided generation, and validating/repairing malformed output
- Streaming responses into ASP.NET Core: Server-Sent Events and SignalR patterns for token-by-token UI updates

Hands-on lab: Extend the /chat endpoint with a tool-calling function that queries a local SQL Server table (e.g., order status), and add a strongly-typed structured-output endpoint that returns a validated C# record instead of free text. Add SSE streaming to the front end.

Deliverable: Tool-enabled, streaming chat API with at least one function call into SQL Server and one structured-output endpoint, covered by unit tests that assert on the deserialized DTO.

Day 5 — Project 1 — SmartSupport API (Build Day)

Focus: Consolidate Week 1 into a shippable slice: a production-shaped ASP.NET Core Web API that answers customer-support questions using company data, with streaming, tool calling, and provider fallback.

Topics covered:

- Architecture review: layering IChatClient behind a clean service boundary (ports & adapters), keeping AI concerns out of controllers
- Error handling and graceful degradation for AI calls: timeouts, retries with Polly, circuit breaking around a flaky/expensive dependency
- Basic cost and token accounting per request, logged via OpenTelemetry
- Containerizing the API with Docker for local parity with later deployment weeks

Hands-on lab: Full build day. Assemble SmartSupport API: ASP.NET Core Web API + Microsoft.Extensions.AI + Azure OpenAI/Foundry Local fallback + SQL Server tool call + streaming SSE endpoint + Dockerfile.

Deliverable: Project 1 (SmartSupport API) complete, containerized, running locally via `docker run`, with a README covering architecture decisions. Presented as a 10-minute walkthrough to the cohort.

Phase 2 — Building: Data & Retrieval (Week 2)

Goal: ground AI responses in real enterprise data through a properly engineered RAG pipeline.

Day 6 — Vector Databases & Embeddings

Focus: Understand embeddings as just another indexed data type, and choose a vector store the way you'd choose any other persistence technology — by consistency, scale, and ops burden.

Topics covered:

- Microsoft.Extensions.VectorData: the vector-store abstraction layer, its record/collection model, and how it mirrors your existing repository pattern
- Comparing vector stores for .NET: Azure AI Search (vector + hybrid), PostgreSQL pgvector, Qdrant, SQL Server native vector support, and in-memory stores for tests
- Embedding generation strategies, dimensionality trade-offs, and re-embedding/versioning when models change
- Similarity search fundamentals: cosine vs dot-product vs Euclidean, and why it matters for ranking quality

Hands-on lab: Stand up a vector store (Azure AI Search or pgvector in Docker) behind Microsoft.Extensions.VectorData, embed a sample document set, and run top-k similarity queries from a .NET console app.

Deliverable: Working vector-store integration with a benchmark note comparing two store options against your organization's likely data volume and compliance constraints.

Day 7 — Data Ingestion Pipelines

Focus: Build the unglamorous but decisive part of RAG: getting messy enterprise documents into clean, chunked, embeddable form.

Topics covered:

- Microsoft.Extensions.DataIngestion: readers, chunkers, and enrichment pipeline stages
- Chunking strategies: fixed-size vs semantic vs structural (headings/tables), and their effect on retrieval quality
- Handling real enterprise inputs: PDFs, Word docs, HTML, Confluence/SharePoint exports, and OCR for scanned content
- Incremental ingestion and freshness: change detection, re-indexing, and avoiding full-corpus rebuilds

Hands-on lab: Build an ingestion pipeline that reads a folder of PDFs/Word docs, chunks them with Microsoft.Extensions.DataIngestion, generates embeddings, and writes into the Day 6 vector store, with an idempotent re-run for changed files.

Deliverable: A reusable ingestion console/worker service, plus a short data-quality report showing chunk counts, average chunk size, and any parsing failures.

Day 8 — RAG Architecture Patterns

Focus: Move from 'it retrieves something' to a RAG pipeline an architect would sign off on: grounded, cited, and evaluable.

Topics covered:

- Naive RAG vs hybrid search (keyword + vector) vs re-ranking pipelines
- Grounding and citation: returning source references alongside generated answers, and refusing to answer when retrieval confidence is low

- Failure modes: stale context, retrieval of irrelevant chunks, and prompt-context overflow — and mitigation patterns
- Introduction to evaluating retrieval quality (precision@k, recall) as a precursor to Week 4's evaluation module

Hands-on lab: Add hybrid search and a re-ranking step to the Day 7 pipeline; wire retrieved chunks with citations into a RAG answer endpoint that refuses to answer outside its grounded context.

Deliverable: A RAG query endpoint that returns answers with source citations and a documented confidence/refusal threshold.

Day 9 — Semantic Kernel Deep Dive

Focus: Use Semantic Kernel where it still earns its keep: plugin composition, memory, and planning on top of the Microsoft.Extensions.AI foundation.

Topics covered:

- Kernel, plugins (native + prompt functions), and the planner model
- Semantic Kernel memory abstractions and how they relate to Microsoft.Extensions.VectorData
- Where Semantic Kernel still fits versus Microsoft Agent Framework — composing skills into a single app vs orchestrating multiple autonomous agents
- Wrapping the Week 2 RAG pipeline as a Semantic Kernel plugin so it can be reused by an agent in Week 3

Hands-on lab: Wrap the RAG endpoint as a Semantic Kernel native plugin; build a kernel-driven chat loop that decides when to call the RAG plugin vs answer directly.

Deliverable: A Semantic Kernel plugin package for RAG retrieval, reusable in later modules, with unit tests for the plugin's function-calling contract.

Day 10 — Project 2 — Enterprise Knowledge Assistant (Build Day)

Focus: Ship a RAG-powered internal knowledge assistant that combines everything from Week 2 into a coherent, demoable application.

Topics covered:

- Architecture review: ingestion service, vector store, RAG API, and a web front end as independently deployable pieces
- Blazor or Razor front end wired to the streaming RAG endpoint, with citation display
- Access control considerations: document-level permissions flowing through to retrieval (don't retrieve what the user can't see)
- Docker Compose for the full stack (API + vector store + SQL Server) for local development

Hands-on lab: Full build day. Assemble the Enterprise Knowledge Assistant: ingestion service + vector store + Semantic Kernel RAG plugin + ASP.NET Core/Blazor front end, docker-composed end to end.

Deliverable: Project 2 (Enterprise Knowledge Assistant) complete and running via `docker compose up`, demoed to the cohort with at least one permission-scoped query example.

Phase 2 — Building: Agentic AI (Week 3)

Goal: move from single-turn tool calling to safely orchestrated multi-agent systems, interoperable via MCP.

Day 11 — Agentic AI Concepts

Focus: Learn to evaluate agentic AI the way you'd evaluate any architectural pattern: what problem it solves, what it costs in complexity and control, and when a simpler pipeline is the right call.

Topics covered:

- Reasoning loops: plan → act → observe, and how this differs from a single tool-calling turn
- Autonomy vs control: guardrails, approval gates, and bounded action spaces
- Agent memory: short-term (conversation), long-term (vector-backed), and procedural (learned tool use)
- An architect's checklist for 'should this be an agent?' — cost, latency, determinism, and auditability trade-offs

Hands-on lab: Design review exercise: take three real workflows from your own organization and score them against the agentic-fit checklist; prototype the reasoning loop for the best candidate in a console app without a framework, to feel the raw mechanics first.

Deliverable: A one-page 'agent fit' scoring rubric applied to three real candidate workflows, plus a minimal hand-rolled plan-act-observe loop.

Day 12 — Microsoft Agent Framework

Focus: Move from hand-rolled loops to Microsoft Agent Framework 1.0 — the production-ready, open-source convergence of Semantic Kernel's model-agnostic core and AutoGen's multi-agent orchestration, with stable, long-term-supported APIs.

Topics covered:

- Agent Framework 1.0 core abstractions: agents, threads, and tools, and its connectors for Microsoft Foundry, Azure OpenAI, OpenAI, and others
- Orchestration patterns: sequential, concurrent, handoff, group chat, and Magentic-One — and which pattern fits which workflow shape
- Graph-based workflows and middleware hooks for cross-cutting concerns (logging, safety checks, cost tracking) across an agent run
- Migrating from Semantic Kernel Agents to Microsoft Agent Framework: what changes, what stays, and when migration is (and isn't) worth it

Hands-on lab: Rebuild the Day 11 hand-rolled loop as a Microsoft Agent Framework agent; give it the Week 2 RAG plugin as a tool and observe how the framework handles planning and tool selection.

Deliverable: A working single-agent Microsoft Agent Framework project with at least one custom tool and framework-managed conversation state.

Day 13 — Model Context Protocol (MCP)

Focus: Use MCP to expose enterprise capabilities as standardized, reusable tools that any compliant agent or IDE can consume — not just your own app.

Topics covered:

- MCP fundamentals: servers, clients, tools, resources, and prompts as the shared vocabulary between agents and systems

- MCP C# SDK v2.0: the 2026-07-28 spec's stateless-by-default architecture, `Mcp-Method`/`Mcp-Name` headers, and why that matters for horizontal scaling
- Building an MCP server on ASP.NET Core (`AddMcpServer().WithHttpTransport()` + `MapMcp()`), and building an MCP client that consumes it
- Multi Round-Trip Requests (MRTR) for tools that need user input without a persistent session, and backward compatibility with v1 clients

Hands-on lab: Build a stateless MCP server on ASP.NET Core exposing two internal tools (e.g., SQL Server order lookup, ticket creation), then connect the Day 12 Agent Framework agent to it as an MCP client.

Deliverable: An MCP server (deployable independently) plus an agent that consumes its tools over MCP rather than in-process function calls.

Day 14 — Multi-Agent Orchestration & Guardrails

Focus: Coordinate multiple specialized agents safely — with explicit handoffs, human approval where it matters, and hard limits on what any agent can do unsupervised.

Topics covered:

- Designing multi-agent topologies: a router/orchestrator agent handing off to specialist agents (e.g., billing, technical support, reporting)
- Human-in-the-loop approval steps for high-risk actions (refunds, data changes) inside an otherwise autonomous workflow
- Guardrails: input/output filtering, action allow-lists, and budget/step limits to prevent runaway agent loops
- Observability for agent runs: tracing multi-step decisions so failures are debuggable, not mysterious

Hands-on lab: Extend Day 13's setup into a two-agent handoff workflow (e.g., a support agent that hands off to a billing agent for refund requests), with a human-approval gate before any refund action executes.

Deliverable: A multi-agent handoff workflow with a working approval gate and a trace/log of a full run showing each decision point.

Day 15 — Project 3 — Agentic Ops Assistant (Build Day)

Focus: Ship a multi-agent operations assistant that combines MCP tool servers, Agent Framework orchestration, and human approval gates into a coherent internal tool.

Topics covered:

- Architecture review: separating MCP tool servers, the orchestrator agent, and the client UI as independently deployable services
- Cost and step budgeting per agent run
- Logging and tracing strategy carried into Week 4's evaluation work
- Containerizing the multi-service agentic system with Docker Compose

Hands-on lab: Full build day. Assemble the Agentic Ops Assistant: MCP tool server(s) + Microsoft Agent Framework orchestrator with handoff + approval gate + minimal UI, docker-composed.

Deliverable: Project 3 (Agentic Ops Assistant) complete and running via `docker compose up`, demoed with one fully-autonomous path and one human-approved path.

Phase 3 — Production: ML.NET, Evaluation & Responsible AI (Week 4)

Goal: apply engineering rigor — testing, safety, cost control — to everything built so far.

Day 16 — ML.NET for Classical ML — Choosing the Right Tool

Focus: Recognize the large class of problems that don't need an LLM at all, and know when to reach for ML.NET's classical machine learning instead of (or alongside) generative AI.

Topics covered:

- When ML.NET beats an LLM: structured tabular prediction, classification, forecasting, and anomaly detection with lower cost and higher determinism
- ML.NET pipeline basics and Model Builder for regression/classification/forecasting scenarios using your own SQL Server data
- ONNX as the interop point: training in ML.NET or exporting from other tools, then serving predictions from the same .NET process as your generative features
- Hybrid architectures: using ML.NET for a fast/cheap first pass (e.g., intent classification or anomaly flagging) before invoking an LLM only when needed

Hands-on lab: Train an ML.NET classification model on a sample support-ticket dataset (e.g., urgency/category) and wire it in front of the SmartSupport API from Project 1 as a pre-filter before the LLM call.

Deliverable: A working ML.NET model integrated as a cost-saving pre-filter, with a short cost/latency comparison of the hybrid pipeline vs an LLM-only pipeline.

Day 17 — AI Evaluation & Regression Testing

Focus: Bring the discipline of unit and regression testing to AI features — because 'it looked right when I tried it' is not a test suite.

Topics covered:

- Microsoft.Extensions.AI.Evaluation: building evaluation datasets, scoring functions, and regression baselines
- Automated evaluation of RAG quality (groundedness, relevance) and agent task success rate
- Human-in-the-loop review workflows for cases automated scoring can't fully judge
- Integrating evaluation runs into the development loop so prompt/model changes get caught before they ship

Hands-on lab: Build an evaluation suite for the Project 2 RAG assistant using Microsoft.Extensions.AI.Evaluation, covering groundedness and answer relevance, and establish a baseline score to guard against regressions.

Deliverable: An automated evaluation suite (runnable via `dotnet test` or a console runner) with a documented baseline and pass/fail thresholds.

Day 18 — Responsible AI, Security & Governance

Focus: Apply the same security-architecture rigor you already bring to auth and data protection to prompt injection, content safety, and AI governance.

Topics covered:

- Prompt injection and jailbreak defenses: input sanitization, tool-permission scoping, and the limits of prompt-based defenses
- Azure AI Content Safety integration for harmful-content filtering on both input and output
- PII handling in prompts, logs, and vector stores; data residency and retention considerations

- Governance for architects: model/prompt change management, audit trails, and responsible-AI review checkpoints

Hands-on lab: Add Azure AI Content Safety checks to the Project 3 agentic pipeline's input and output, and write an injection test suite that attempts to override the system prompt or trigger unauthorized tool calls.

Deliverable: A content-safety-hardened pipeline plus a red-team test report documenting attempted injection attacks and outcomes.

Day 19 — Performance & Cost Engineering

Focus: Treat token spend and model latency as first-class capacity-planning concerns, the way you already treat database load and API throughput.

Topics covered:

- Caching strategies for AI calls: semantic caching, prompt-level caching, and cache invalidation trade-offs
- Batching and request shaping to reduce round trips and token overhead
- Model routing: small/cheap models (or Foundry Local) for easy cases, escalating to larger models only when needed
- Setting and enforcing token/cost budgets per request and per tenant, with alerting

Hands-on lab: Add a semantic cache and a small-model-first routing strategy (Foundry Local or a smaller Azure OpenAI deployment) to Project 1, and measure the resulting cost/latency delta under a simulated load test.

Deliverable: A load-test report (before/after) showing cost and p95 latency improvements from caching and model routing.

Day 20 — Checkpoint — Evaluation & Safety Pass Across All Projects

Focus: Consolidate Week 4 by applying evaluation, safety, and cost controls consistently across Projects 1–3 before moving into production deployment.

Topics covered:

- Cross-project review: apply the Day 17 evaluation pattern to Projects 1 and 3, not just Project 2
- Apply Day 18 content-safety and injection defenses to all three projects
- Apply Day 19 caching/routing where it makes sense across the portfolio
- Peer review: cohort members review each other's evaluation and safety coverage

Hands-on lab: Checkpoint lab. Retrofit evaluation suites, content-safety checks, and cost controls across all three projects; run the full evaluation suite for each and record baselines.

Deliverable: All three projects passing their own evaluation suite with documented baselines, ready for the production-hardening work in Week 5.

Phase 3 — Production: DevOps, Deployment & Capstone (Week 5)

Goal: ship. Take every project from a laptop to a monitored, CI/CD-deployed Azure service.

Day 21 — Containerizing AI Applications

Focus: Apply container best practices you already know, adapted for the specific footprint of AI workloads — model artifacts, secrets, and GPU/NPU dependencies.

Topics covered:

- Multi-stage Dockerfiles for .NET AI apps: keeping build tools out of the runtime image
- Externalizing configuration and secrets (never bake API keys into images); managed identity in containerized Azure deployments
- Image size and cold-start considerations for services that call out to LLMs vs services that host local models (Foundry Local)
- Local multi-service orchestration with Docker Compose as a rehearsal for Azure Container Apps/AKS

Hands-on lab: Harden the Dockerfiles for all three projects: multi-stage builds, non-root users, externalized secrets via environment variables mapped to Key Vault references.

Deliverable: Production-shaped Dockerfiles for all three projects, passing a basic container security lint.

Day 22 — Azure Deployment for AI Workloads

Focus: Take the containerized projects from your laptop to Azure, using the same deployment targets you'd choose for any .NET microservice.

Topics covered:

- Azure Container Registry: build, tag, push, and image vulnerability scanning
- Choosing between Azure Container Apps and AKS for AI workloads: scale-to-zero, GPU node pools, and cost implications
- Provisioning Azure OpenAI/Microsoft Foundry resources as part of the environment, with infrastructure as code (Bicep or Terraform overview)
- Networking and identity: private endpoints, managed identity to Azure OpenAI/Key Vault, and outbound egress control

Hands-on lab: Push Project 1's image to Azure Container Registry and deploy it to Azure Container Apps with managed identity access to Azure OpenAI and Key Vault, defined via a Bicep template.

Deliverable: Project 1 running live on Azure Container Apps, deployed from an IaC template checked into source control.

Day 23 — CI/CD with Azure DevOps & Azure Pipelines

Focus: Extend your existing CI/CD instincts to the extra artifacts AI apps carry: prompts, evaluation suites, and model/version pins.

Topics covered:

- Azure Pipelines stages for AI apps: build, unit test, run the Microsoft.Extensions.AI.Evaluation suite as a quality gate, containerize, push to ACR, deploy
- Prompt and configuration versioning: treating prompt templates as reviewable, diffable artifacts, not string literals scattered in code

- Blue-green and canary rollout patterns for model or prompt changes, so a regression is caught on a slice of traffic, not everyone
- Approval gates and environment promotion (dev → staging → prod) for AI features, including a manual responsible-AI sign-off step

Hands-on lab: Build an Azure Pipelines YAML pipeline for Project 1 that runs its evaluation suite as a gate, builds and pushes the image to ACR, and deploys to Azure Container Apps with a manual approval before production.

Deliverable: A working Azure Pipelines definition (checked into the repo) that fails the build on evaluation regressions and requires approval before production deploy.

Day 24 — Monitoring & Observability for AI Systems

Focus: Instrument AI features with the same rigor as any production service — plus the AI-specific signals that don't show up in a standard APM dashboard.

Topics covered:

- Application Insights/Azure Monitor integration via OpenTelemetry for AI calls: latency, error rate, and token usage as tracked metrics
- Tracing multi-step agent runs so a failed workflow can be replayed and debugged, not just flagged
- Dashboarding cost, latency, and quality drift over time (linking back to Week 4's evaluation baselines)
- Alerting thresholds: cost spikes, latency degradation, evaluation-score drift, and content-safety trigger rates

Hands-on lab: Instrument Project 3 (the agentic system) with OpenTelemetry tracing across the full multi-agent run, and build an Azure Monitor dashboard showing cost, latency, and evaluation-score trend lines.

Deliverable: A live monitoring dashboard for Project 3 with at least one configured alert rule, plus a trace of one full multi-agent run.

Day 25 — Capstone — AI-Augmented Enterprise Platform (Presentation Day)

Focus: Bring every phase together in a single, end-to-end, production-deployed system, presented as you would present a design to a real architecture review board.

Topics covered:

- Capstone architecture: SmartSupport-style API + RAG knowledge assistant + agentic ops workflow + ML.NET pre-filter, unified behind one deployed platform
- Full CI/CD pipeline running the evaluation gate, deploying to Azure Container Apps
- Live monitoring dashboard and at least one documented incident/response scenario (simulated cost spike or evaluation regression)
- Retrospective: what you'd change architecturally with more time, and a 90-day plan for applying this to a real system at work

Hands-on lab: Final integration and deployment. Merge Projects 1–3 into the unified Capstone platform, deploy via the Week 5 pipeline, and rehearse the presentation.

Deliverable: Capstone platform deployed live on Azure with monitoring active; a 15-minute architecture presentation and live demo to the cohort and instructor panel.

Assessment & Certification Criteria

Participants are assessed on delivered, working artifacts rather than written exams — appropriate for an audience that already knows how to build software.

Component	Weight	Criteria
Project 1 — SmartSupport API	15%	Streaming chat API with tool calling, provider fallback, containerized
Project 2 — Enterprise Knowledge Assistant	20%	Grounded RAG with citations, access-scoped retrieval, docker-composed
Project 3 — Agentic Ops Assistant	20%	Multi-agent handoff with MCP tools and a working human-approval gate
Week 4 Checkpoint	15%	Evaluation suites, content-safety hardening, and cost controls applied across all projects
Capstone platform & presentation	30%	Unified, live-deployed system with CI/CD gate and monitoring dashboard; clarity of architecture presentation

A participant who completes all five components at a passing standard receives a Certificate of Completion; exceptional Capstone work (clean architecture, strong evaluation coverage, thoughtful responsible-AI handling) is recognized with distinction.

Resource Library

Curated references for self-study and post-course deepening, current as of August 2026:

Microsoft.Extensions.AI & the .NET AI Ecosystem

- Develop .NET apps with AI features — Microsoft Learn (learn.microsoft.com/dotnet/ai/overview)
- .NET + AI ecosystem tools and SDKs — Microsoft Learn (learn.microsoft.com/dotnet/ai/dotnet-ai-ecosystem)
- Microsoft.Extensions.AI libraries reference — Microsoft Learn (learn.microsoft.com/dotnet/ai/microsoft-extensions-ai)
- Introducing Microsoft.Extensions.AI — .NET Blog (devblogs.microsoft.com/dotnet)

Microsoft Agent Framework & Semantic Kernel

- Microsoft Agent Framework documentation and migration guide — devblogs.microsoft.com/agent-framework
- Microsoft Ships Production-Ready Agent Framework 1.0 for .NET and Python — Visual Studio Magazine (April 2026)
- Migrating from Semantic Kernel Agents to Microsoft Agent Framework — Microsoft .NET Blog / community guides

Model Context Protocol

- Official MCP C# SDK — github.com/modelcontextprotocol/csharp-sdk
- MCP C# SDK reference documentation — csharp.sdk.modelcontextprotocol.io
- Announcing v2.0 of the official MCP C# SDK — .NET Blog (devblogs.microsoft.com/dotnet)

Microsoft Foundry & Foundry Local

- What is Foundry Local? — Microsoft Learn (learn.microsoft.com/azure/foundry-local)
- Foundry Local is now Generally Available — Microsoft Foundry Blog (devblogs.microsoft.com/foundry)
- Foundry Local SDK Reference — Microsoft Learn

ML.NET & Evaluation

- ML.NET documentation and Model Builder — dotnet.microsoft.com/apps/ai/ml-dotnet
- Microsoft.Extensions.AI.Evaluation — Microsoft Learn / .NET Blog
- Azure AI Content Safety documentation — learn.microsoft.com/azure/ai-services/content-safety

Deployment & Operations

- Azure Container Apps documentation — learn.microsoft.com/azure/container-apps
- Azure Pipelines documentation — learn.microsoft.com/azure/devops/pipelines
- Application Insights for OpenTelemetry — learn.microsoft.com/azure/azure-monitor

Note: this is a fast-moving stack. Microsoft.Extensions.AI, Microsoft Agent Framework, and the MCP C# SDK all ship frequent updates — re-check the linked Microsoft Learn and .NET Blog pages before each cohort run for API or package-name changes.